Relatório – Testes Automatizados do Truco Gaudério

Disciplina de Qualidade de Software 2026/02 · Renato e Lucas · Nov 11, 2026 · @Renato

Introdução

Este relatório documenta, etapa por etapa, os testes automatizados com **PyTest** que aplicamos ao jogo *Truco Gaudério com CBR*, escrito em Python. O jogo coloca um humano contra um bot que decide suas jogadas por raciocínio baseado em casos (CBR), usando os 100 vizinhos mais próximos numa base de partidas.

Decisão de abordagem: testamos o sistema **como ele foi entregue, sem corrigir o código do jogo**. Quando um teste revela um defeito, ele é marcado como `xfail` (falha esperada), com o motivo descrito. Assim a suíte continua verde e serve como registro dos defeitos encontrados.

Item	Detalhe
Sistema sob teste	Pacote `truco/`, com 12 módulos: carta, baralho, jogador, bot, jogo, truco, envido, flor, interface, dados, cbr e __main__
Ferramenta	PyTest, com pytest-cov para medir a cobertura
Base de casos da IA	`dbtrucoimitacao_maos.csv`, com cerca de 14 MB
Entregas	Documento de requisitos e casos de teste, `RenatoLucastestesoftware.zip` e `RenatoLucastestesoftware.mp4` (de 5 a 15 min)
Prazo	As apresentações começam em 16/11/2026 e a entrega final é em 23/11/2026, às 14h30

Tipos de teste exigidos no enunciado

Tipo de teste	Obrigatório?
Validação de requisitos e cenários de uso	Sim
Chamadas de função com parâmetros variados	Sim
Entradas de dados do usuário	Sim

Tipo de teste	Obrigatório?
Retornos de função	Sim
IA implementada no jogo	Sim
Exceções descritas no código	Não
Mensagens de erro	Não
Início e fim de laços	Não
Combinações de `if`s encadeados	Não

Etapas do trabalho

1. Preparação do ambiente de testes
2. Requisitos e casos de teste
3. Testes unitários por módulo
4. Testes da IA (CBR)
5. Testes de entradas do usuário e cenários ponta a ponta
6. Execução, cobertura e relatório de defeitos
7. Empacotamento e roteiro do vídeo

Etapa 1 – Preparação do ambiente de testes

Resultado: o ambiente PyTest está pronto. Os 22 testes de fumaça rodam em cerca de 4 segundos: 21 passaram e 1 falhou de propósito, comprovando o defeito D01. A cobertura inicial é de 38% do pacote `truco`.

Estrutura criada no projeto

```
trabalho_qualidade_software/
├── .venv/                    ambiente virtual Python (não vai para o Git)
├── configurar_ambiente.bat cria o .venv, instala as dependências e roda os
testes
├── pytest.ini                configuração do PyTest e dos marcadores
├── requirements-test.txt     pandas, scikit-learn, pytest e pytest-cov
├── .gitignore                ignora .venv, cache, cobertura e o jogadas.csv
├── truco/                    código do jogo (não alterado)
└── tests/
    ├── README.md             como rodar e como ler o resultado
    ├── conftest.py           fixtures compartilhadas
    └── test_00_ambiente.py testes de fumaça do ambiente
```

As dependências ficam isoladas num **ambiente virtual** `.venv` dentro do projeto. Assim as versões de pandas, scikit-learn e pytest usadas nos testes não se misturam com o Python do sistema, e qualquer integrante do grupo consegue recriar o mesmo ambiente.

Configuração (pytest.ini)

- `--strict-markers` faz o PyTest recusar marcadores digitados errado.
- `xfail_strict = true` serve de alarme: se um defeito marcado como `xfail` deixar de acontecer, o teste aparece como XPASS e falha.
- Os marcadores separam a suíte por categoria do enunciado. Assim dá para rodar só uma parte durante o vídeo.

Marcador	Uso	Exemplo de comando
`requisito(id)`	Liga o teste a um requisito ou caso de teste do documento	`pytest -m requisito`
`ia`	Testes da IA (CBR)	`pytest -m ia`
`entrada`	Entradas do usuário simuladas	`pytest -m entrada`
`cenario`	Partida completa executada pelo `__main__`	`pytest -m cenario`
`defeito(id)`	Comprova um defeito do jogo, sempre junto com `xfail`	`pytest -m defeito`
`lento`	Carrega a base de casos completa	`pytest -m "not lento"`
`sem_adaptador`	Roda como o usuário, sem o contorno do D01	—

Fixtures do conftest.py

As fixtures permitem testar o jogo sem mexer no código dele. Cada teste recebe só as peças de que precisa.

Fixture	O que faz	Para que tipo de teste
`nova_carta`, `nova_mao`	Criam cartas a partir de texto, como "1 de ESPADAS"	Todos
`baralho_fixo`	Baralho que entrega as cartas numa ordem conhecida	Requisitos e retornos
`jogador_com_mao`, `bot_com_mao`	Jogador e bot com mãos definidas pelo teste	Funções com parâmetros variados e `if`s
`entradas_usuario`	Simula o teclado (`input()`) e registra cada pergunta feita	Entradas do usuário e laços de validação
`cbr_falsa`	IA "dublê" com respostas controladas e registro das chamadas	Truco, Envido, Flor e Bot isolados
`cbr_real`	IA verdadeira com a base de 27.515 casos, carregada uma vez por execução	Testes da IA
`executar_partida`	Roda o `truco/__main__.py` inteiro com entradas, semente e mãos fixas	Cenários ponta a ponta
`interface`	Instância da classe Interface	Mensagens na tela (com `capsys`)

Todo teste roda numa pasta temporária. Isso evita que o arquivo `jogadas.csv`, que o jogo grava ao fim de cada mão, suje o projeto.

Se o jogo pedir mais entradas do que o teste forneceu, a simulação do teclado lança `EntradasEsgotadas`. Dessa forma um laço infinito de validação vira um erro que o teste consegue verificar, em vez de travar a suíte.

Defeito encontrado: D01 – o jogo não inicia

Ao montar o ambiente, descobrimos que `python -m truco` falha logo no início com `ValueError: Usecols do not match columns`.

Campo	Detalhe
Local	`truco/dados.py`, método `tratamento_inicial_df` (linha 12)
Causa	A base `dbtrucoimitacao_maos.csv` é separada por **tabulação**, mas o `pd.read_csv` usa o separador padrão, que é a vírgula. Conferimos no Git que o arquivo versionado também usa tabulação.
Impacto	Crítico: nenhuma partida pode ser jogada e a IA não carrega
Teste que comprova	`test_D01_jogo_carrega_base_de_casos_sem_adaptador`, que aparece como XFAIL
Contorno nos testes	Um adaptador de leitura no `conftest.py` detecta o separador. O código do jogo não foi alterado.

Testes de fumaça (test_00_ambiente.py)

Teste	O que verifica	Resultado
`test_todos_os_modulos_do_pacote_importam` (12 casos)	Cada módulo do pacote importa sem erro	Passou
`test_arquivos_de_dados_existem`	A base de casos e o modelo de registro estão presentes	Passou
`test_fabricas_de_carta_e_mao`	Cartas criadas a partir de texto	Passou
`test_baralho_fixo_entrega_cartas_na_ordem`	O baralho controlado entrega as cartas na ordem pedida	Passou
`test_bot_com_mao_fixa_calcula_atributos`	O bot classifica a mão em Alta, Média e Baixa e calcula a qualidade	Passou

Teste	O que verifica	Resultado
`test_simulacao_de_teclado`	O `input()` simulado responde e avisa quando as entradas acabam	Passou
`test_cbr_falsa_registra_chamadas`	A IA dublê devolve uma resposta por chamada	Passou
`test_ia_real_carrega_base_de_casos`	A IA real carrega mais de 20 mil casos, sem nenhum valor vazio	Passou
`test_partida_inicia_e_distribui_cartas`	A partida pede os dois nomes e dá 3 cartas a cada jogador	Passou
`test_partida_com_maos_fixas`	O cenário consegue fixar as mãos do humano e do bot	Passou
`test_D01_jogo_carrega_base_de_casos_sem_adaptador`	O jogo, rodado como o usuário roda, carrega a base de casos	XFAIL, comprova o D01

Como reproduzir

O jeito mais rápido é dar um duplo clique em `configurar_ambiente.bat`. Para fazer manualmente no Windows:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements-test.txt
pytest -v
pytest --cov=truco --cov-report=html
```

O último comando gera o relatório de cobertura em `htmlcov/index.html`. A cobertura inicial ficou em 38% e vai crescer nas próximas etapas.

Etapa 2 – Requisitos e casos de teste

Resultado: levantamos 18 requisitos funcionais e 3 não funcionais, 9 cenários de uso e 44 casos de teste. Cada caso de teste aponta para um requisito e para uma categoria do enunciado. Esta seção é a entrega "documento de requisitos e cenários de uso".

Os requisitos vêm de três fontes: as regras do README, que seguem o site Jogatina, o comportamento descrito nas docstrings do código e os comentários do autor. Quando o README não define uma regra, como os pontos de um envido recusado, adotamos a regra implementada e a marcamos como "regra do sistema".

Requisitos funcionais

ID	Requisito	Fonte
RF01	O baralho é o espanhol, com 40 cartas: 1 a 7, 10, 11 e 12 dos naipes Espadas, Ouros, Copas e Bastos. Sem 8, 9 e curingas.	README
RF02	Cada jogador recebe 3 cartas, distribuídas de forma automática e aleatória.	README
RF03	A força das cartas, da menor para a maior, é 4, 5, 6, 7, 10, 11, 12, 1, 2, 3. As manilhas valem mais que todas, nesta ordem: 7 de Ouros, 7 de Espadas, 1 de Bastos e 1 de Espadas.	README e pontos.py
RF04	Na rodada vence a carta mais forte. Cartas de mesma força empatam.	README (definição de "Empatar")
RF05	A mão é disputada em melhor de 3 rodadas. Quem vence 2 rodadas ganha o valor da aposta, que é 1 ponto se ninguém pediu truco.	README
RF06	Quem vence uma rodada joga primeiro na rodada seguinte.	jogo.py
RF07	O truco eleva a mão a 2 pontos, o retruco a 3 e o vale 4 a 4. Quem recusa entrega a quem pediu o valor anterior (1, 2 ou 3). O mesmo jogador não pode aumentar a aposta duas vezes seguidas.	README e truco.py
RF08	Ao ir ao baralho, o jogador desiste da mão e o adversário ganha o valor atual da aposta.	README

ID	Requisito	Fonte
RF09	O envido só pode ser pedido na primeira rodada, com as 3 cartas na mão. Envido vale 2 pontos, Real Envido vale 5 e Falta Envido vale 12 menos os pontos do adversário. Recusar dá a quem pediu 1, 2 ou 5 pontos, respectivamente (regra do sistema).	README e envido.py
RF10	Os pontos de envido de uma mão são 20 mais a soma de duas cartas do mesmo naipe. Sem naipe repetido, valem os pontos da maior carta. As figuras 10, 11 e 12 valem 0.	Planilha de codificação e bot.py
RF11	No envido vence quem tem mais pontos. Em caso de empate vence o jogador 1, que é o mão (regra do sistema).	envido.py
RF12	Quem tem 3 cartas do mesmo naipe pode cantar Flor, que vale 3 pontos e bloqueia o envido. Se os dois têm flor, há Contraflor, que vale 6, ou Contraflor e o Resto.	README e flor.py
RF13	A partida termina quando um jogador chega a 12 pontos.	__main__.py
RF14	A cada jogada o jogador vê suas cartas e as opções válidas: [4] Truco, [5] Flor (só com flor), [6] Envido, [7] Real Envido, [8] Falta Envido (só com 3 cartas) e [9] Ir ao baralho.	jogador.py
RF15	Uma entrada inválida não encerra o jogo: o sistema mostra "Selecione um valor válido!" e pergunta de novo.	__main__.py
RF16	O bot decide qual carta jogar e como responder ao truco, ao envido e à flor usando CBR, a partir dos casos mais parecidos com a situação atual.	README e cbr.py
RF17	A IA usa a base `dbtrucoimitacao_maos.csv` e consulta os 100 vizinhos mais próximos.	README, cbr.py e comentários do autor
RF18	Ao fim de cada mão, o sistema registra as jogadas no arquivo `jogadas.csv`.	dados.py

Requisitos não funcionais

ID	Requisito	Fonte
RNF01	O jogo inicia com `python -m truco`, tendo pandas e scikit-learn instalados.	README
RNF02	Cada decisão da IA leva menos de 1 segundo, para que o jogo não pareça travado.	Critério do grupo
RNF03	Ao começar uma nova mão, todo o estado da mão anterior é zerado: cartas, rodadas, apostas, envido e flor.	Docstrings dos métodos `resetar`

Cenários de uso

ID	Cenário	Fluxo principal	Fluxos alternativos	Requisitos
UC01	Iniciar partida	O jogador informa o seu nome e o do bot, e cada um recebe 3 cartas.	Nome vazio.	RF01, RF02, RNF01
UC02	Jogar uma mão	O humano escolhe uma carta, o bot responde e quem vence a rodada joga primeiro. Após 2 vitórias o placar é atualizado e uma nova mão é dada.	O bot é o primeiro a jogar.	RF03 a RF06, RNF03
UC03	Humano pede truco	O humano escolhe [4] e o bot aceita, recusa ou aumenta.	Retruco e vale 4.	RF07, RF16
UC04	Bot pede truco	O bot pede truco e o humano responde [0] Recusar, [1] Aceitar ou [2] Aumentar.	Resposta inválida.	RF07, RF15
UC05	Envido	Um jogador pede [6], [7] ou [8] e o outro responde. O vencedor recebe os pontos.	Recusa, aumento para Real ou Falta Envido.	RF09 a RF11

ID	Cenário	Fluxo principal	Fluxos alternativos	Requisitos
UC06	Flor	O jogador com 3 cartas do mesmo naipe canta [5] e ganha 3 pontos.	Os dois têm flor e há Contraflor.	RF12
UC07	Ir ao baralho	O humano escolhe [9], o bot ganha o valor da aposta e uma nova mão é dada.	—	RF08
UC08	Entrada inválida	O humano digita uma letra ou um número fora das opções, o sistema avisa e pergunta de novo.	—	RF15
UC09	Fim de partida	Um jogador chega a 12 pontos, o sistema anuncia o vencedor e encerra.	—	RF13

Etapa 3 – Testes unitários

Resultado: implementamos os casos CT01 a CT30 em 7 arquivos. A suíte completa tem agora **288 testes aprovados e 27 falhas esperadas**, e roda em cerca de 5 segundos. A cobertura do pacote `truco` subiu de 38% para **75%**. Encontramos **10 defeitos novos** (D08 a D17), o que leva o total a 17.

Os casos CT31 a CT34 (entradas na tela principal e ir ao baralho) dependem da partida inteira rodando. Por isso foram para a Etapa 5, junto com os cenários de uso.

Arquivos e resultados

Arquivo	Casos de teste	Aprovados	Falhas esperadas	Defeitos comprovados
test_baralho_carta.py	CT01, CT02, CT03, CT05, CT06	90	2	D08
test_jogo.py	CT04, CT07, CT08, CT09	23	3	D02

Arquivo	Casos de teste	Aprovados	Falhas esperadas	Defeitos comprovados
test_envido.py	CT10 a CT17	41	7	D03, D04, D09, D10
test_truco.py	CT18 a CT23	18	8	D05, D06, D07, D09, D11, D12
test_flor.py	CT25 a CT27	23	2	D09, D14
test_jogador_bot.py	CT24 e métodos do Jogador e do Bot	40	2	D13, D15
test_interface.py	CT28 a CT30	32	2	D16, D17
test_00_ambiente.py (Etapa 1)	CT44	21	1	D01
Total		**288**	**27**	**17 defeitos**

Um teste parametrizado conta uma vez para cada combinação de parâmetros. Por isso o número de testes é maior que o de funções de teste.

Técnicas usadas

- **Parametrização (`@pytest.mark.parametrize`):** cobre as chamadas com parâmetros variados e as combinações de `if`s. Exemplos: os pontos das 40 cartas, uma por uma; os quatro tipos de confronto entre cartas (manilha ou comum de cada lado); as opções 0 a 3 do envido.
- **Oráculo independente:** a força esperada das cartas foi escrita no teste a partir do README, sem importar `pontos.py`. O teste `test_carta_vencedora_todos_os_pares_de_forca_diferente` confere sozinho os 1.560 confrontos possíveis do baralho.
- **Limites de laço:** o baralho é criado por `for n in range(1, 13)` com `if n < 8 or n >= 10`. Testamos os números 1, 7, 8, 9, 10, 12 e 13, ou seja, o início, o fim e as bordas do intervalo excluído. Nos laços `while` de validação testamos uma, duas e três respostas inválidas antes da válida, contando quantas vezes a pergunta foi feita.
- **Dublê de IA (`cbr_falsa`):** nos testes de truco, envido e flor, a resposta do bot é fixada pelo teste. Assim cada ramo dos `if`s é exercitado de propósito, e o teste também confere quais parâmetros o jogo passou para a IA. Foi assim que achamos o D10.

- **Teclado simulado (`entradas_usuario`):** troca o `input()` por uma lista de respostas e guarda cada pergunta exibida, para conferir o texto das perguntas.
- **Captura da tela (`capsys`):** compara as mensagens impressas, inclusive a moldura do placar, caractere por caractere.
- **Exceções (`pytest.raises`):** comprovam o `IndexError` ao tirar carta de um baralho vazio ou ao jogar um índice fora da mão.
- **Falha esperada estrita (`xfail` com `xfail_strict`):** cada defeito vira um teste que descreve o comportamento correto. Se alguém corrigir o jogo, o teste passa a aparecer como XPASS e avisa que a marcação deve ser removida.

Exemplo de teste parametrizado com um caso de defeito, em `test_envido.py`:

```
CASOS_ENVIDO = [
    pytest.param(MAO_33, 33, id="par-7-6-mesmo-naipe"),
    pytest.param(MAO_7, 7, id="naipes-diferentes"),
    pytest.param(("12 de COPAS", "7 de COPAS", "4 de OUROS"), 27,
                 id="figura-e-7-mesmo-naipe",
                 marks=[pytest.mark.defeito("D03"),
                        pytest.mark.xfail(reason="D03: par com figura vira
'maior carta'")]),
]

@pytest.mark.requisito("CT10")
@pytest.mark.parametrize("classe", [Jogador, Bot])
@pytest.mark.parametrize("mao, pontos", CASOS_ENVIDO)
def test_calcula_envido(nova_mao, classe, mao, pontos):
    assert classe("Teste").calcula_envido(nova_mao(*mao)) == pontos
```

Novos defeitos encontrados

ID	Defeito	Onde	Teste que comprova	Gravidade
D08	Com três cartas de mesma força (por exemplo, três 12), a mesma carta é classificada como Alta e Baixa. O bot quebra com `ValueError` ao receber essa mão.	carta.py, `classificar_carta`; bot.py, `calcular_qualidade_mao`	test_classificar_tres_cartas_de_mesma_forca e test_bot_recebe_tres_cartas_de_mesma_forca	Alta: derruba o jogo
D09	Uma resposta não numérica, como "sim", nas perguntas de truco, envido ou flor gera `ValueError` e encerra o jogo	truco.py, envido.py e flor.py: `int(input(...))`	test_entrada_nao_numerica_* (3 arquivos)	Alta: derruba o jogo

ID	Defeito	Onde	Teste que comprova	Gravidade
D10	Para decidir se está perdendo, o bot usa os próprios pontos de envido no lugar do placar do adversário	envido.py, chamadas a `avaliar_envido`	test_bot_empatado_no_placar_nao_esta_perdendo	Média: a IA decide com um dado errado
D11	O estado "vale_quatro" nunca é gravado. Depois do vale 4 um novo pedido é aceito e rebaixa a mão para 3 pontos.	truco.py, `pedir_vale_quatro`	test_depois_do_vale_4_nenhum_pedido_e_aceito	Média
D12	A mensagem "Jogador X aceitou o pedido" cita quem pediu, e não quem aceitou	truco.py	test_mensagem_cita_quem_aceitou	Baixa: só a mensagem
D13	`Jogador.resetar` não zera `pediu_flor`. Na mão seguinte o envido do bot fica bloqueado.	jogador.py	test_jogador_resetar_zera_pediu_flor	Média
D14	A Contraflor e o Resto vale só 3 pontos, menos que a Contraflor (6)	flor.py, `contraflor_resto`	test_contraflor_e_resto_vale_mais_que_a_contraflor	Média
D15	O bot nunca canta flor: `criar_mao` já marca `flor = True`, e por isso a condição de cantar nunca é verdadeira	bot.py, `jogar_carta`	test_bot_com_flor_canta_flor	Média
D16	A opção [5] Flor some a partir da segunda vez que as opções são exibidas	jogador.py, `mostrar_opcoes`	test_flor_continua_visivel_na_segunda_exibicao	Baixa
D17	Com 1 carta na mão, a opção [4] Truco não é exibida, embora a regra permita pedir truco na última rodada	jogador.py, `mostrar_opcoes`	test_truco_disponivel_na_ultima_rodada	Baixa

Atualização dos casos da Etapa 2: o CT22 confirmou o D12 e o CT24 confirmou o D13, que estavam como "defeito previsto". Todos os outros casos de CT01 a CT30 foram executados. Os que não estão ligados a um defeito passaram.

Cobertura por módulo

Módulo	Etapa 1	Etapa 3
baralho.py	78%	100%

Módulo	Etapa 1	Etapa 3
jogador.py	81%	100%
pontos.py	100%	100%
interface.py	28%	99%
jogo.py	36%	99%
carta.py	66%	97%
envido.py	29%	97%
truco.py	17%	94%
flor.py	19%	91%
bot.py	54%	89%
dados.py	40%	40%
cbr.py	39%	39%
__main__.py	25%	25%
Total	**38%**	**75%**

Os três módulos que ainda não subiram são a IA (`cbr.py` e `dados.py`) e o laço principal (`__main__.py`). Eles são o alvo das Etapas 4 e 5.

Casos de teste

O resultado esperado segue o requisito, e não o que o código faz hoje. Na coluna Situação, "Defeito confirmado" indica que já executamos o caso à mão e o jogo falhou. "Defeito previsto" indica uma suspeita vinda da leitura do código, que será confirmada nas próximas etapas.

As categorias usam estas siglas: REQ (requisitos e cenários), PAR (chamadas com parâmetros variados), ENT (entrada do usuário), RET (retorno de função), IA, EXC (exceções), MSG (mensagens), LAÇO e IF.

ID	Req.	Categoria	Entrada / situação	Resultado esperado	Arquivo de teste	Situação
CT01	RF01	REQ, LAÇO	Criar um baralho	40 cartas, 10 por naipe, sem 8 nem 9 e sem repetição	test_baralho_carta.py	A executar
CT02	RF02	PAR, LAÇO	Dar a mão ao humano e ao bot	3 cartas para cada um e 34 cartas no baralho	test_baralho_carta.py	A executar
CT03	RF02	RET	Embaralhar com sementes diferentes	As mesmas 40 cartas em outra ordem	test_baralho_carta.py	A executar

ID	Req.	Categoria	Entrada / situação	Resultado esperado	Arquivo de teste	Situação
CT04	RF03	IF, PAR	Todos os pares entre manilha e manilha, manilha e comum, comum e manilha, comum e comum	Vence a carta mais forte pela tabela do RF03	test_jogo.py	A executar
CT05	RF03	RET	`retornar_pontos_carta` para as 40 cartas	Manilhas valem 52, 50, 42 e 40. As comuns vão de 24 (o 3) a 1 (o 4).	test_baralho_carta.py	A executar
CT06	RF03	IF, RET	`classificar_carta` com mãos variadas, com e sem manilha	Classificação Alta, Média e Baixa correta	test_baralho_carta.py	A executar
CT07	RF04	IF	3 de Copas contra 3 de Ouros	Empate	test_jogo.py	Defeito confirmado: sempre vence a carta do jogador 1
CT08	RF05	RET	`adicionar_rodada` com o vencedor 1, o vencedor 2 e um ganhador inválido	Retorna 1 ou 2 e soma uma rodada ao vencedor. Com ganhador inválido retorna "Erro".	test_jogo.py	A executar
CT09	RF06	IF	`quem_joga_primeiro` após vitória do humano e após vitória do bot	O vencedor fica com `primeiro = True`	test_jogo.py	A executar
CT10	RF10	PAR, RET	`calcula_envido` com 7+6 de Copas, naipes todos diferentes, 12+7 de Copas e 10+11 de Copas	33, 7, 27 e 20	test_envido.py	Defeito confirmado: com figura retorna 7 e 4
CT11	RF09	ENT, LAÇO	O humano responde ao envido do bot com 0, 1, 2 e 3, e com "9" seguido de "1"	Cada opção é aceita. A opção inválida faz repetir a pergunta.	test_envido.py	A executar
CT12	RF09	REQ	Envido recusado e envido aceito	Recusado: quem pediu ganha 1. Aceito: o vencedor ganha 2.	test_envido.py	A executar
CT13	RF09	REQ	Real Envido aceito e recusado	O vencedor ganha 5. Na recusa, quem pediu ganha 2.	test_envido.py	A executar
CT14	RF09	REQ	Falta Envido com o adversário em 0 e em 8 pontos	Vale 12 e 4 pontos. A recusa dá 5 a quem pediu.	test_envido.py	A executar
CT15	RF11	IF	Pontos de envido: J1 maior, J2 maior e empate	Vence J1, J2 e J1, respectivamente	test_envido.py	A executar
CT16	RF09	IF, RET	Segundo pedido de envido na mesma mão e pedido de jogador bloqueado	`controlador_envido` retorna None e não muda os pontos	test_envido.py	A executar
CT17	RF09	REQ, EXC	O bot escolhe envido na vez dele (6, 7 ou 8)	O envido acontece normalmente	test_envido.py	Defeito confirmado: TypeError por chamada com argumentos errados
CT18	RF07	REQ	Truco aceito	A mão passa a valer 2	test_truco.py	Defeito confirmado: continua valendo 1
CT19	RF07	REQ, IF	Truco, retruco e vale 4, todos aceitos	A mão vale 3 e depois 4	test_truco.py	Defeito confirmado: o vale 4 nunca é alcançado
CT20	RF07	REQ	Recusas de truco, retruco e vale 4	Quem pediu ganha 1, 2 e 3 pontos	test_truco.py	A executar
CT21	RF07	IF, RET	O mesmo jogador pede truco duas vezes seguidas, ou pede após o vale 4	`controlador_truco` retorna None	test_truco.py	A executar
CT22	RF07	ENT, LAÇO, MSG	O humano responde ao truco do bot com 0, 1, 2 e com uma opção inválida	A pergunta mostra o valor da mão e repete na opção inválida. A mensagem cita quem aceitou.	test_truco.py	Defeito previsto: a mensagem cita quem pediu, e não quem aceitou

ID	Req.	Categoria	Entrada / situação	Resultado esperado	Arquivo de teste	Situação
CT23	RF07	RET	`retornar_valor_aposta` e `retornar_quem_fugiu`	Retornam 1 e o número do jogador	test_truco.py	Defeito confirmado: `retornar_quem_fugiu` retorna o próprio método
CT24	RNF03	RET	`resetar` de Truco, Envido, Flor, Jogador e Bot	Todos os atributos voltam ao valor inicial	test_jogador_bot.py	Defeito previsto: o Jogador não zera `pediu_flor`
CT25	RF12	PAR, IF	`checa_flor` com 3 cartas do mesmo naipe e com 2+1	True e False	test_flor.py	A executar
CT26	RF12	REQ	Só o humano tem flor, só o bot tem e os dois têm (Contraflor aceita ou recusada)	+3 para quem tem flor, +6 para o vencedor da Contraflor	test_flor.py	A executar
CT27	RF12	ENT, LAÇO	`decisao_jogador` com "5" e depois "1"	Repete a pergunta e depois retorna True	test_flor.py	A executar
CT28	RF14	MSG, IF	`mostrar_opcoes` com 3, 2 e 1 cartas, com e sem flor	Opções exibidas de acordo com o RF14	test_interface.py	A executar
CT29	RF14	MSG	`border_msg`, placar, vencedor do envido e vencedor da flor	Texto e moldura corretos	test_interface.py	A executar
CT30	RF14	MSG, IF	`desenhar_cartas` para os 4 naipes	Símbolos ♠ ♦ ♥ ♣ corretos	test_interface.py	A executar
CT31	RF15	ENT, EXC	Na escolha de carta, o humano digita "a"	Aviso e nova pergunta	test_entradas_usuario.py	Defeito previsto: ValueError encerra o jogo
CT32	RF15	ENT, EXC	Com 3 cartas na mão, o humano digita "3"	Aviso e nova pergunta	test_entradas_usuario.py	Defeito previsto: IndexError encerra o jogo
CT33	RF08	REQ, ENT	O humano escolhe [9] Ir ao baralho	O bot ganha 1 ponto e uma nova mão é dada	test_entradas_usuario.py	A executar
CT34	UC01	ENT, MSG	Nomes "Renato" e "Lucas", e nome vazio	Os nomes aparecem nas mensagens e o nome vazio não quebra o jogo	test_entradas_usuario.py	A executar
CT35	RF17	IA	Carregar a base de casos	27.515 casos, sem vazios, com as mesmas colunas do registro	test_cbr.py	A executar
CT36	RF16	IA, RET, PAR	`jogar_carta` com mãos variadas nas rodadas 1, 2 e 3	Retorna um índice válido da mão	test_cbr.py	A executar
CT37	RF16	IA, RET	`truco` com qualidade de mão muito baixa, média e alta	Retorna 0, 1 ou 2. Com mão fraca foge (0) e com mão forte não foge.	test_cbr.py	A executar
CT38	RF16	IA, RET	`envido` para cada tipo e para o pedido do próprio bot	Tipo 6 retorna de 0 a 3, tipos 7 e 8 retornam 0 ou 1, o pedido do bot retorna 6, 7 ou 8	test_cbr.py	A executar
CT39	RF16, RNF02	IA	A mesma situação consultada duas vezes	A mesma decisão, em menos de 1 s	test_cbr.py	A executar
CT40	RF16	IA, IF	Rodada 1 em `jogar_carta`	Consulta a coluna da primeira carta do robô	test_cbr.py	Defeito previsto: a rodada 1 consulta a coluna da terceira carta
CT41	RF18	REQ	`Dados` grava as cartas da segunda e da terceira rodada	Cada carta é gravada na coluna certa	test_dados.py	Defeito previsto: colunas trocadas e nome com maiúscula errada
CT42	UC02	REQ, cenário	Partida com mãos fixas em que o humano vence 2 rodadas	O humano ganha 1 ponto, o placar aparece e uma nova mão é dada	test_cenarios.py	A executar
CT43	UC09	REQ, cenário, LAÇO	Um jogador chega a 12 pontos	A mensagem "... ganhou o jogo" aparece e o laço principal termina	test_cenarios.py	A executar
CT44	RNF01	REQ, EXC	`python -m truco`	O jogo inicia	test_00_ambiente.py	Defeito confirmado (D01)

Cobertura das categorias do enunciado

Categoria do enunciado	Casos de teste
Requisitos e cenários de uso (obrigatório)	CT01, CT12 a CT14, CT17 a CT20, CT26, CT33, CT41 a CT44
Chamadas com parâmetros variados (obrigatório)	CT02, CT04, CT10, CT25, CT36
Entradas do usuário (obrigatório)	CT11, CT22, CT27, CT31 a CT34
Retornos de função (obrigatório)	CT03, CT05, CT06, CT08, CT10, CT16, CT21, CT23, CT24, CT36 a CT38
IA do jogo (obrigatório)	CT35 a CT40
Exceções	CT17, CT31, CT32, CT44
Mensagens de erro e de tela	CT22, CT28 a CT30, CT34
Início e fim de laços	CT01, CT02, CT11, CT22, CT27, CT43
Combinações de `if`s	CT04, CT06, CT07, CT09, CT15, CT16, CT19, CT21, CT25, CT28, CT30, CT40

Defeitos registrados até aqui

ID	Defeito	Caso de teste	Situação
D01	A base de casos é separada por tabulação e o jogo a lê com vírgula, por isso não inicia	CT44	Confirmado
D02	Cartas de mesma força não empatam: vence sempre a carta do jogador 1	CT07	Confirmado
D03	O envido de duas cartas do mesmo naipe com figura é calculado errado	CT10	Confirmado
D04	O bot pede envido com os argumentos errados (TypeError)	CT17	Confirmado
D05	Truco aceito não muda o valor da mão, que continua em 1	CT18	Confirmado
D06	Depois do retruco, um novo pedido chama retruco de novo, e o vale 4 nunca acontece	CT19	Confirmado

ID	Defeito	Caso de teste	Situação
D07	`retornar_quem_fugiu` retorna o próprio método, e não um valor	CT23	Confirmado